Using Embedding Layer – in TF/Keras

Bhupen Sinha

25-07-2024

TABLE OF CONTENTS

EMBEDDING LAYERS

CONTEXT

In natural language processing (NLP) and many machine learning applications, data often comes in the form of text.

However, machine learning models, especially deep learning models, work with numerical data.

Therefore, it's essential to convert text data into a numerical form. This transformation is crucial for capturing the semantic meaning and relationships between words, which traditional one-hot encoding or simple bag-of-words representations cannot fully capture.

This is where **embedding layers** come into play.

WHAT ARE EMBEDDING LAYERS?

An **embedding layer** is a type of neural network layer that converts categorical data, specifically words in NLP, into dense vectors of fixed size.

These vectors are learned representations of the words, where similar words have similar vector representations.

Embeddings provide a way to represent words in a continuous vector space, making it possible to capture syntactic and semantic properties of words.

KEY CHARACTERISTICS OF EMBEDDING LAYERS:

1. **Dimensionality Reduction**: Embeddings reduce the dimensionality of word representations compared to one-hot encoding, which can be sparse and high-dimensional.
2. **Capturing Semantics**: By learning from the context in which words appear, embeddings can capture semantic relationships. For instance, in a good embedding space, the vector for "king" may be similar to the vector for "queen."
3. **Efficient Computation**: The dense vectors produced by embedding layers make computation more efficient and manageable for large datasets.

EXAMPLES

Example 1: Word Embeddings in NLP

In NLP tasks like sentiment analysis, text classification, and language translation, words are converted into embeddings before being fed into the model.

For instance, in a sentiment analysis task, sentences like "I love this product" and "This product is amazing" might have words like "love" and "amazing" mapped to vectors close to each other in the embedding space, indicating their similar sentiment.

Example 2: Word2Vec and GloVe

Word2Vec and **GloVe** are popular word embedding techniques used to pre-train word vectors. These vectors can be directly used in various NLP tasks or fine-tuned for specific applications.

For example, in Word2Vec, similar words have similar vectors, enabling the model to understand relationships like "king - man + woman = queen."

Example 3: Embedding Layers in Other Domains

Beyond NLP, embedding layers are used in recommender systems.

For example, in a movie recommendation system, an embedding layer can represent users and movies in the same vector space, where the distance between a user vector and a movie vector can indicate the user's preference for that movie.

WHAT DOES EMBEDDING LAYER LOOK LIKE

STRUCTURE AND FUNCTIONALITY

1. **Input**: The input to an embedding layer consists of integer indices. These indices correspond to words or categories that have been pre-processed and converted into numerical form.

2. **Embedding Matrix**: The core component of the embedding layer is the **embedding matrix**, which is a dense matrix of shape (vocab_size, embedding_dim).
    - vocab_size: The total number of unique words or categories in the dataset, including a possible padding token.
    - embedding_dim: The size of each word vector (the number of features in the embedding space).

3. **Output**: For each input index, the embedding layer outputs the corresponding dense vector (row) from the embedding matrix.
    - These vectors represent the features of the input data in a lower-dimensional space.

VISUAL REPRESENTATION

Imagine a simplified example with a vocabulary size of 5 (including a padding token) and an embedding dimension of 3.

The embedding matrix might look something like this:

Index	word/ token	Embedding Vector
0	data	[0.1, 0.2, 0.3]
1	engineering	[0.5, -0.1, 0.6]
2	for	[0.4, 0.8, -0.2]
3	time	[-0.3, 0.7, 0.5]
4	series	[0.2, -0.5, 0.9]

- **Index**: The row index represents the integer encoding of a word or category.
- **Embedding Vector**: The vector represents the dense, learned representation of the word or category in the embedding space.

HOW IT WORKS IN PRACTICE

When an input sequence like [2, 3, 4] is passed through the embedding layer, it fetches the corresponding rows from the embedding matrix:

- For index 2, the vector [0.4, 0.8, -0.2] is retrieved.
- For index 3, the vector [-0.3, 0.7, 0.5] is retrieved.
- For index 4, the vector [0.2, -0.5, 0.9] is retrieved.

Thus, the output for the input sequence [2, 3, 4] would be a 2D array (matrix) of shape (3, 3):

```
[[ 0.4,  0.8, -0.2],
 [-0.3,  0.7,  0.5],
 [ 0.2, -0.5,  0.9]]
```

Classification example

Let us take a sample dataset

```
# Define the sentences
sentences = [
    "I love machine learning",
    "Deep learning is fascinating",
    "Natural language processing is a challenge",
    "I enjoy learning new things",
    "Machine learning models are powerful"
]

# Define the binary labels
labels = [1, 1, 0, 0, 1]
```


Applying Keras's tokenization, words are encoded as below: -

{'learning': 1, 'i': 2, 'machine': 3, 'is': 4, 'love': 5, 'deep': 6, 'fascinating': 7, 'natural': 8, 'language': 9, 'processing': 10, 'a': 11, 'challenge': 12, 'enjoy': 13, 'new': 14, 'things': 15, 'models': 16, 'are': 17, powerful': 18}

The size of the vocabulary is = total number of unique words = 18

With these token sequences, the sentences would like:

Sentence	Encoded version of the sentence
"I love machine learning"	[2, 5, 3, 1]
"Deep learning is fascinating"	[6, 1, 4, 7]
"Natural language processing is a challenge"	[8, 9, 10, 4, 11, 12]
"I enjoy learning new things"	[2, 13, 1, 14, 15]
"Machine learning models are powerful"	[3, 1, 16, 17, 18]

Using Embedding Layer – in TF/Keras

Padding

To handle sentences of varying lengths, we use padding to ensure they all have a uniform length. This process involves adding extra tokens, usually zeros, to shorter sentences so they match the length of the longest sentence in the dataset.

The uniformity is crucial for efficient batch processing and compatibility with neural networks.

max_length = 7 # Maximum length of the sentences after padding

Sentence	Encoded version of the sentence
"I love machine learning"	[2, 5, 3, 1, 0, 0, 0]
"Deep learning is fascinating"	[6, 1, 4, 7, 0, 0, 0]
"Natural language processing is a challenge"	[8, 9, 10, 4, 11, 12, 0]
"I enjoy learning new things"	[2, 13, 1, 14, 15, 0, 0]
"Machine learning models are powerful"	[3, 1, 16, 17, 18, 0, 0]

KERAS MODEL BUILDING

Below is illustrative example of how Keras's embedding layer transforms the input token to 8-dimensional vector representation. The objective is to covert the encoded words (integers) into N-dimensional dense vector representation. (N = 8 in this case) – example below.

Input (word/ token)		Embedding layer transforms input to 8 dim vectors
Token	**Encoded Value**	**8-Dimensional Embedding (Random Floats)**
learning	1	[0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89]
i	2	[0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15]
machine	3	[0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48]
is	4	[0.48, -0.19, 0.29, -0.42, 0.55, -0.21, 0.34, -0.67]
love	5	[0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22]
deep	6	[0.38, -0.52, 0.34, -0.29, 0.77, -0.46, 0.29, -0.54]
fascinating	7	[0.15, -0.26, 0.62, -0.38, 0.83, -0.17, 0.47, -0.59]
natural	8	[0.24, -0.57, 0.49, -0.68, 0.64, -0.33, 0.58, -0.41]
language	9	[0.53, -0.11, 0.39, -0.72, 0.74, -0.49, 0.66, -0.13]
processing	10	[0.43, -0.66, 0.52, -0.31, 0.57, -0.22, 0.36, -0.47]
a	11	[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]
challenge	12	[0.19, -0.53, 0.74, -0.45, 0.38, -0.68, 0.52, -0.36]
enjoy	13	[0.32, -0.37, 0.65, -0.29, 0.48, -0.17, 0.72, -0.58]
new	14	[0.22, -0.29, 0.71, -0.37, 0.58, -0.61, 0.49, -0.31]
things	15	[0.47, -0.39, 0.55, -0.21, 0.61, -0.42, 0.34, -0.65]
models	16	[0.51, -0.24, 0.63, -0.46, 0.39, -0.19, 0.45, -0.38]
are	17	[0.13, -0.58, 0.43, -0.49, 0.62, -0.28, 0.54, -0.32]
powerful	18	[0.28, -0.34, 0.76, -0.39, 0.53, -0.44, 0.42, -0.57]

But why do need to convert text data (words) into dense vector representation?

1. Dimensionality Reduction

- **Problem:** Raw text data is high-dimensional, with a unique dimension for each word or character in the vocabulary. This results in sparse representations, where most dimensions are zero, leading to inefficiencies.
- **Solution:** Dense vectors represent words in a lower-dimensional space, capturing the essential information and reducing the computational load.

2. Semantic Meaning

- **Problem:** One-hot encoding or sparse representations do not capture the semantic meaning or relationships between words. For instance, "king" and "queen" are semantically related, but their one-hot vectors would be orthogonal and lack this information.
- **Solution:** Dense vectors capture semantic similarities between words. In a well-trained embedding space, similar words have similar vectors. For example, "king" and "queen" might be close in the embedding space, reflecting their related meanings.

3. Efficient Computation

- **Problem:** Sparse vectors are inefficient for computation, especially in operations like similarity measures, clustering, or feeding data into neural networks.
- **Solution:** Dense vectors allow for more efficient computation and storage, enabling faster and more scalable model training and inference.

4. Transfer Learning

- **Problem:** Building models from scratch for different NLP tasks can be time-consuming and data-intensive.
- **Solution:** Pre-trained dense vector representations, like Word2Vec, GloVe, or BERT, can be used across various tasks. These embeddings encapsulate general linguistic information learned from large corpora and can be fine-tuned for specific tasks, leading to better performance with less data.

5. Handling Out-of-Vocabulary Words

- **Problem:** In one-hot encoding, unseen words are problematic as they have no predefined representation.
- **Solution:** Dense vector spaces can generalize to unseen words by placing them in contextually appropriate positions, providing a form of understanding even for words not seen during training.

6. Compatibility with Neural Networks

- **Problem:** Neural networks require numerical input and are sensitive to the quality of the input representation.
- **Solution:** Dense vector representations are compatible with neural networks and provide a meaningful, continuous input space that neural networks can exploit to learn complex patterns in data.

Configuring the Keras model

- **Input data**
  - 5 samples (sentence)
    - Each sentence is of length 7 words (tokens, after padding)
    - Each sentence has a label of either 0 or 1

```
# Create the model
model = Sequential()

model.add(Embedding(input_dim    = vocab_size,     # 18
                    output_dim   = embedding_dim,  # 8
                    input_length= max_length))    # 7

model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))
```

- **Embedding layer**
  - Takes a batch of sentences (batch size = 1)

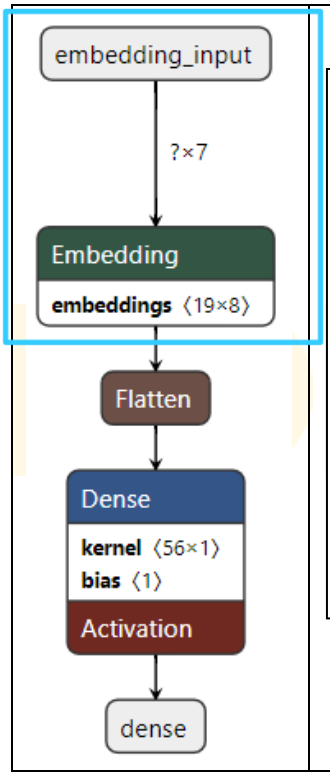
Sentence	Encoded version of the sentence
"I love machine learning"	[2, 5, 3, 1, 0, 0, 0]

- Each of the encoded value (for a word) will be mapped to a row in the embedding table

Input (word/ token)		Embedding layer transforms input to 8 dim vectors
Token	**Encoded Value**	**8-Dimensional Embedding (Random Floats)**
learning	**1**	[0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89]
i	**2**	[0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15]
machine	**3**	[0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48]
is	4	[0.48, -0.19, 0.29, -0.42, 0.55, -0.21, 0.34, -0.67]
love	**5**	[0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22]
deep	6	[0.38, -0.52, 0.34, -0.29, 0.77, -0.46, 0.29, -0.54]
fascinating	7	[0.15, -0.26, 0.62, -0.38, 0.83, -0.17, 0.47, -0.59]
natural	8	[0.24, -0.57, 0.49, -0.68, 0.64, -0.33, 0.58, -0.41]
language	9	[0.53, -0.11, 0.39, -0.72, 0.74, -0.49, 0.66, -0.13]
processing	10	[0.43, -0.66, 0.52, -0.31, 0.57, -0.22, 0.36, -0.47]
a	11	[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]
challenge	12	[0.19, -0.53, 0.74, -0.45, 0.38, -0.68, 0.52, -0.36]
enjoy	13	[0.32, -0.37, 0.65, -0.29, 0.48, -0.17, 0.72, -0.58]
new	14	[0.22, -0.29, 0.71, -0.37, 0.58, -0.61, 0.49, -0.31]
things	15	[0.47, -0.39, 0.55, -0.21, 0.61, -0.42, 0.34, -0.65]
models	16	[0.51, -0.24, 0.63, -0.46, 0.39, -0.19, 0.45, -0.38]
are	17	[0.13, -0.58, 0.43, -0.49, 0.62, -0.28, 0.54, -0.32]
powerful	18	[0.28, -0.34, 0.76, -0.39, 0.53, -0.44, 0.42, -0.57]
padding	0	[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]

- Output of the embedding layer for the batch of samples (1 sample in this case) will be

Input (word/ token)		Embedding layer transforms input to 8 dim vectors
Token	**Encoded Value**	**8-Dimensional Embedding (Random Floats)**
learning	1	[0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89]
i	2	[0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15]
machine	3	[0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48]
love	5	[0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22]
padding	0	[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]




```
[
[0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15],  # i
[0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22],  # love
[0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48],  # machine
[0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89]   # learning
[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]   # padding /0
[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]   # padding /0
[0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]   # padding /0
]
```

Shape of array is 7x8

For the batch, the shape of array is 1x7x8

- **Array Shape**: 7x8
    - This means there are 7 sequences (or time steps) and each sequence has a vector of 8 dimensions.
- **Batch Shape**: 1x7x8
    - For a batch size of 1, the shape would be 1x7x8, where:
        - 1 represents the batch size (one sample in the batch).
        - 7 represents the number of sequences or time steps.
        - 8 represents the dimensionality of each sequence vector.

In summary, if you have a batch size of 1 and each sample has a sequence of 7 vectors each of size 8, then the batch shape would indeed be 1x7x8.

- **Flatten** the output of embedding layer

  - Why?

    - Dense layers (fully connected layers) expect a one-dimensional input for each sample.
    - The embedding layer produces a multi-dimensional output, such as a 2D sequence of vectors.
    - Flattening converts this 2D output into a 1D vector, making it suitable for dense layers.

  - **Example Workflow**

    1. **Embedding Layer Output**:
       1. Suppose you have an embedding layer that outputs a sequence of vectors, with each vector representing a word in a sentence.
       2. The output shape might be batch_size x sequence_length x embedding_dim.

       Example:

       - Input Sentence: "i love machine learning"
       - Shape after embedding: 1x7x8 (batch size of 1, 7 time steps, 8-dimensional embeddings)

    2. **Flattening**:
       1. Apply the flatten() function to convert the 1x7x8 output into a 1x56 vector.

       Example:

       - Flattened Output Shape: 1x56

    3. **Dense Layers**:
       1. Feed the flattened vector into dense layers to perform classification, regression, or other tasks.

In our present case,

```
[
    [
        [0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15],  # sequence 1
        [0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22],  # sequence 2
        [0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48],  # sequence 3
        [0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89],  # sequence 4
        [0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24],  # sequence 5 (padding)
        [0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24],  # sequence 6 (padding)
        [0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24]   # sequence 7 (padding)
    ]
]
```

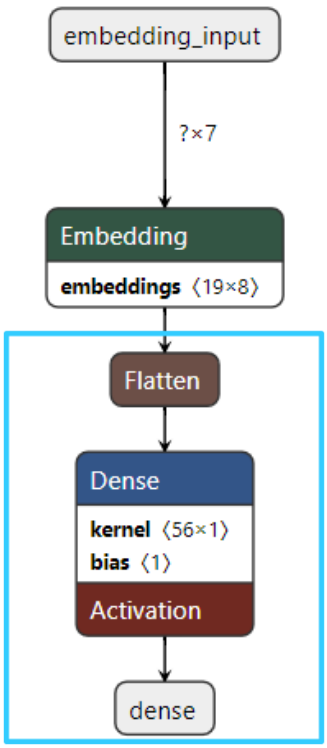
Flattening the above output from the embedding layers gives :-

```
[
    0.05, -0.44, 0.21, -0.62, 0.73, -0.32, 0.48, -0.15,  # sequence 1
    0.26, -0.31, 0.57, -0.63, 0.49, -0.18, 0.71, -0.22,  # sequence 2
    0.33, -0.25, 0.19, -0.55, 0.62, -0.13, 0.76, -0.48,  # sequence 3
    0.12, -0.34, 0.56, -0.78, 0.91, -0.23, 0.67, -0.89,  # sequence 4
    0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24,  # sequence 5 (padding)
    0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24,  # sequence 6 (padding)
    0.11, -0.43, 0.28, -0.56, 0.85, -0.39, 0.61, -0.24   # sequence 7 (padding)
]
```

Shape After Flattening:

1x56 (since 7 sequences × 8 dimensions per sequence = 56 elements)

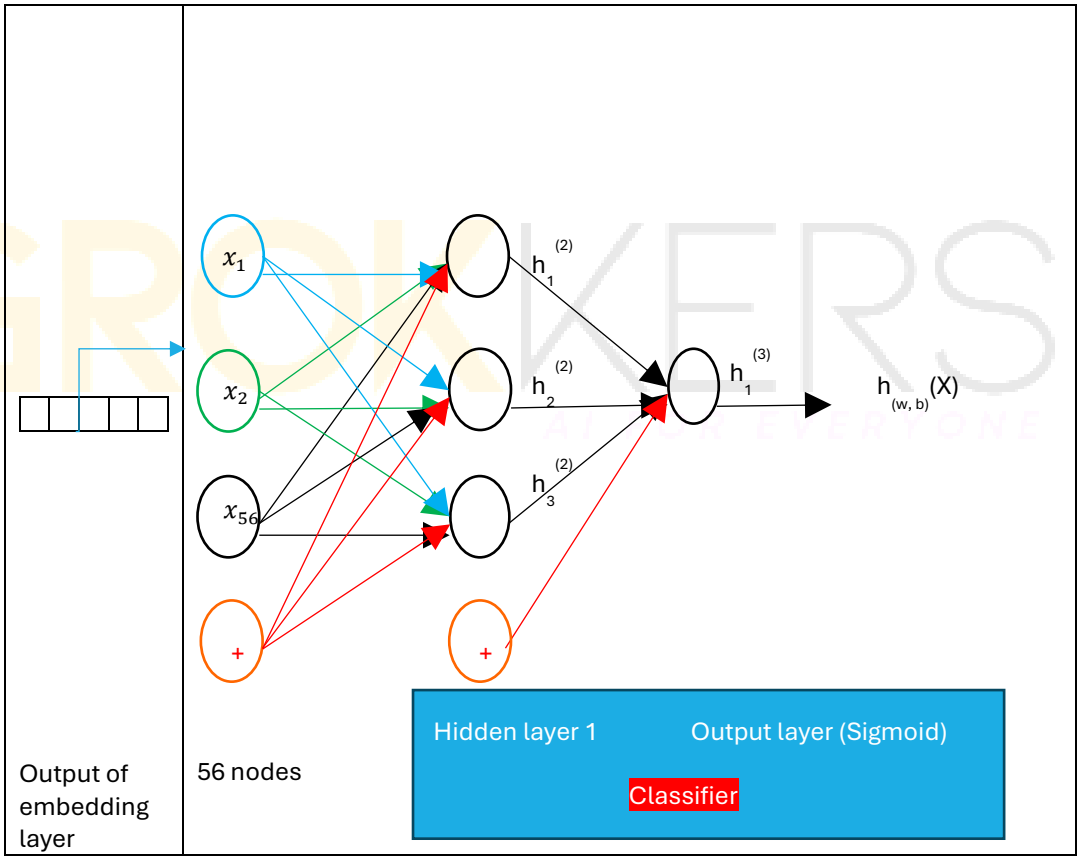
As a one-dimensional array, this shape indicates a single row with 56 columns (or elements)



- **Connection to Further Layers**

  If you connect this flattened vector to a dense layer with a sigmoid activation function, here's what happens:

1. **Flattening**: The output shape of 1x7x8 from the embedding layer becomes 1x56 after flattening.
2. **Dense Layer**: Suppose you add a dense layer with, for example, 10 units and a sigmoid activation function. Each of the 56 nodes (features) in the flattened vector will be connected to each of the 10 units in the dense layer.
3. **Activation Function**: The sigmoid activation function applied to the output of the dense layer will produce values between 0 and 1. This is often used for binary classification tasks.



EMBEDDING LAYER IN FORWARD PASS

Embedding Lookup:

- During forward propagation, each token (e.g., a word or subword) in the input sequence is mapped to an index in the embedding matrix.
- The embedding layer performs a lookup in the embedding table (matrix) to retrieve the dense vector representation for each token.
- For example, if the token index is 3, the embedding layer will fetch the 3rd row from the embedding matrix.

Output:

- The result of this lookup is a matrix where each row corresponds to the embedding vector of a token in the input sequence.
- This matrix is then passed through subsequent layers of the network (e.g., RNNs, CNNs, attention mechanisms, etc.) for further processing.

Example Forward Pass

- **Input Sequence**: [2, 5, 3] (indices of tokens)
- **Embedding Matrix** (assuming dimensions 10x8, where 10 is the vocabulary size and 8 is the embedding dimension):

```
[[0.1, -0.2, 0.3, ...],  # Embedding for index 0
 [0.4, 0.1, -0.5, ...],  # Embedding for index 1
 [0.3, -0.3, 0.4, ...],  # Embedding for index 2
 ...,
 [0.6, -0.2, 0.1, ...]]  # Embedding for index 5
```

- **Embedding Lookup**: Fetches vectors for indices 2, 5, and 3.

```
[[0.3, -0.3, 0.4, ...],  # Embedding for index 2
 [0.6, -0.2, 0.1, ...],  # Embedding for index 5
 [0.5, 0.2, -0.1, ...]]  # Embedding for index 3
```

EMBEDDING LAYER IN BACKWARD PROPAGATION

Backward Propagation

1. **Gradient Calculation**:
    - During backward propagation, gradients of the loss with respect to the output are computed.
    - These gradients are propagated backward through the network, including through the embedding layer.
2. **Updating Embeddings**:
    - The gradient of the loss with respect to each embedding vector is calculated.
    - The embedding table (matrix) is updated using these gradients. This adjustment modifies the vector representations of the tokens to better capture semantic relationships based on the error signal.
    - Typically, an optimizer (e.g., Adam) is used to update the embedding vectors in the embedding matrix.

Example Backward Pass

- **Loss Calculation**: Suppose the model's prediction is compared to the true label, and a loss is computed.
- **Gradient Computation**: Gradients of the loss with respect to each token's embedding vector are computed.
- **Update Embedding Matrix**: The embedding vectors in the matrix are adjusted according to the gradients to minimize the loss in future iterations.

WHERE IS THE EMBEDDING INFORMATION SAVED?

Saving and Loading a Model

1. **Saving the Model**:
    - Model architecture (including the embedding layer structure)
    - Model weights (including the weights of the embedding layer)
    - Training configuration (e.g., optimizer settings)
    - State of the optimizer (if included)
2. **Loading the Model**:

When you load the model, the embedding layer and its weights are restored exactly as they were during training. This means that the learned embeddings are preserved and can be used directly for inference or further training.

What Gets Saved

- **Embedding Layer**: The embedding matrix (table) is stored in the model weights. This matrix contains the learned vector representations for all tokens.
- **Weights**: All weights of the model, including those of the embedding layer, are saved. This ensures that the model's learned parameters are fully preserved.

HOW THE EMBEDDING LAYER / TABLE USED AT THE TIME OF PREDICTION

Using the Embedding Layer During Prediction

1. **Tokenization and Indexing**:
    - **Input Text**: The raw text data is first tokenized into tokens (e.g., words or subwords).
    - **Index Mapping**: Each token is then mapped to an index based on the vocabulary used during training. This mapping is typically done using a tokenizer that was trained alongside the model.
2. **Embedding Lookup**:
    - **Index Lookup**: During prediction, the indices of the tokens (obtained from the tokenization step) are used to look up their corresponding dense vectors from the embedding matrix.
    - **Embedding Matrix**: The embedding layer's weights (embedding matrix) are used to fetch the dense vector for each token index. This matrix was learned and saved during training and is restored when the model is loaded.
3. **Forward Pass**:
    - **Embedding Transformation**: The indices are converted into their dense vector representations using the embedding layer.
    - **Subsequent Layers**: These dense vectors are then passed through the rest of the model (e.g., RNNs, CNNs, attention mechanisms) to perform the prediction task (e.g., classification, regression).

USEFUL LINKS AND REFERENCES

-

-

INDEX

No index entries found.